THORNS

Game Design Document

Computer Games Development | CW208 | ear IV

Student Name	Edward Norton
Student Number	C00286807
Supervisor	Omer Ali
Project Head	Martin Harrigan
Institution	SETU Carlow
Date of Submission	13th April 2026
Academic Year	2025-2026

Contents

1. Acknowledgements 4
2. Game Overview 4
3. Feature Set 5
3.1: Core Features 5
3.2: Gameplay Features 5
3.3: Features Not in Scope 6
4. Gameplay 7
4.1: Core Loop 7
4.2: Controls 7
4.3: Survival Stats 7
4.4: Victory and Defeat 8
5. The Game World 8
5.1: Overview 8
5.2 The Physical World 8
5.2.1: Overview 8
5.2.2: Key Locations 8
5.2.3: Travel 8
5.2.4: Scale 8
5.2.5: Objects 9
5.2.6: Weather and Day/Night 9
5.2.7: Time 9
6. Rendering System 9
6.1: Overview 9
6.2: 2D Rendering 9
6.3: Camera 9
6.3.1: Overview 9
6.3.2: Camera Clamping 9
6.3.3: UI View 10
7. Game Engine 10
7.1: Overview 10
7.2: State Management 10

7.3: Collision Detection 10
7.4: Procedural Generation Pipeline 10
8. The World Layout 11
8.1: Overview 11
8.1.1: Tile Terrain Types 11
8.1.2: POI Exclusion Zones 11
9. Game Characters 11
9.1: Overview 11
9.2: The Player 11
9.3: Enemies 12
10. User Interface 12
10.1: Overview 12
10.2: HUD 12
10.3: Inventory 13
10.4: Menus 13
11. Weapons 14
11.1: Overview 14
12. Musical Scores and Sound Effects 14
12.1: Overview 14
13. Single-Player Game 15
13.1: Overview 15
13.1.1: Story 15
13.1.2: Hours of Gameplay 15
13.1.3: Victory Conditions 15
14. Character Rendering 15
14.1: Overview 15
14.1.1: Player Rendering 15
14.1.2: World Object Rendering 15
15. References 17

1. Acknowledgements

I would like to thank my supervisor Omer Ali for his feedback and guidance throughout this project, and Martin Harrigan for overseeing the module. I would also like to thank the other supervisors, Amr Abdelhafez, Ben O'Shaughnessy, Libor Zachoval, and Noel O'Hara, for their time and constructive feedback during project presentations.

The following third-party resources were instrumental in the development of this project:

- Robert Nystrom: Game Programming Patterns (gameprogrammingpatterns.com), a primary architectural reference for patterns used throughout the codebase.
- Ken Perlin: Improved Noise algorithm, the basis for the Perlin noise implementation used in procedural object placement.
- Robert Bridson: Fast Poisson Disk Sampling in Arbitrary Dimensions, the algorithm used for Voronoi site generation.
- The SFML development team: SFML 3.0 graphics and windowing library.
- The tmxlite library contributors: TMX map parsing used for collision shape loading.

2. Game Overview

Thorns is a top-down 2D survival game developed in C++17 using the SFML 3.0 graphics library. The player finds themselves trapped in an endlessly looping procedurally generated forest, with no memory of how they arrived. Scattered across the world are points of interest, abandoned farms, hidden caches, and a crumbling hideout, each potentially holding the parts needed to construct an escape.

The game was created as a fourth-year academic project for the Computer Games Development programme at SETU Carlow. Its primary purpose is to demonstrate a robust, pattern-driven software architecture using techniques drawn directly from game programming literature, including the Component, State, Flyweight, Spatial Partition, Command, and Update Method patterns. Gameplay is secondary to architecture quality as an assessment criterion, though care has been taken to establish a coherent and extensible gameplay foundation.

Key questions the game answers for the player:

- What is the game? A single-player top-down 2D survival and escape game set in a procedurally generated forest world.
- Where does the game take place? A 128x128-tile (8192x8192 pixel) procedurally generated forest world rebuilt fresh each session from a random seed.
- What do I control? A single character viewed from above, moved with keyboard input, aimed with the mouse.
- What is the focus? Explore the world, manage survival stats (health, stamina, hunger, water), visit points of interest, and collect the components needed to escape.
- What is different? The world is never the same twice. Voronoi-based regionalization combined with Poisson disk sampling and Perlin noise creates a unique map layout on every run.

3. Feature Set

3.1: Core Features

- Procedurally generated world using Voronoi diagram regionalization and Poisson disk sampling for site placement.
- Perlin noise based organic object placement (trees, roots, forest debris).
- Component based player entity with health, stamina, hunger, and water survival stats displayed in the HUD.
- Polygon based collision detection for world objects using the Separating Axis Theorem (SAT).
- Flyweight pattern for shared collision shape data across all world object instances.
- Stack-based game state manager supporting push/pop transitions (Main Menu, Playing, Paused, Settings).
- Keyboard and gamepad input handled through a unified InputController with runtime key rebinding.
- Inventory system with 10 item slots rendered as an in-game panel.
- Custom cursor rendering in screen space, replacing the OS cursor during gameplay.
- Map regeneration at runtime with a new seed (R key during play, removed).
- Enemy system with two enemy types (SavageEnemy, ChomperEnemy) using Object Pool management and line-of-sight AI.
- Day timer HUD: countdown display in the top-right corner; map regenerates and resets when the timer expires.
- Weapons: Knife, Axe, and Gun available as world items with melee and ranged hit detection via EnemyManager.
- Consumable items: Food, Water, First Aid Kit, and Bandage restore survival stats on use from the inventory.

3.2: Gameplay Features

- Survival stat drain: hunger and water drain passively over time; stamina drains when sprinting and regenerates at rest.
- Points of Interest (POIs): Farms and a Player Hideout are placed at Voronoi sites across the map.
- Player movement states: Idle, Walk, Sprint, and Inventory Open (player cannot move while inventory is open).
- Mouse-aimed rotation: the player character always faces the cursor position.
- * Two enemy types patrol the world. SavageEnemy uses a three-state AI (Idle, Chase, Lost). ChomperEnemy adds a Leap attack state triggered at close range.
- Enemy spawning is seeded from Voronoi site positions, distributing enemies across the map each run.
- Combat: melee weapons (Knife, Axe) hit enemies within a short radius; ranged (Gun) performs ray-based hit detection against enemies along the aim vector.
- Item drop: items can be dropped from the inventory back into the world at the player's position.

- Day timer: a visible countdown drives urgency. When it expires, the map regenerates, resetting enemies and loot.

3.3: Features Not in Scope

Feature	Previous Status	Current Status
Enemy AI and combat	In development	Implemented: SavageEnemy and ChomperEnemy with full AI and EnemyManager
Weapons	Planned future feature	Implemented: Knife, Axe, Gun as collectible world items with hit detection
Consumable items beyond Food/Water	Not listed	Implemented: First Aid Kit and Bandage added as stat-restoring consumables
Day timer	Partially noted in 5.2.7	Implemented: DayTimerComponent with HUD panel and map regeneration callback
Item drop	Not listed	Implemented: items can be dropped from inventory back into the world
Save and load	Not in scope	Remains not in scope
Multiplayer	Not in scope	Remains not in scope
Audio	Planned	Remains not implemented
Win condition / escape mechanic	Planned	Planned, not yet implemented

4. Gameplay

4.1: Core Loop

The player spawns at the Hideout at the centre of the procedurally generated map. From there the loop is:

1. Explore: navigate the forest, managing stamina by balancing sprint use with recovery.
2. Survive: keep hunger and water above zero; health drains if either reaches empty.
3. Collect: visit POIs (Farms, Hideout) to gather items and escape components.
4. Escape: assemble the required parts and trigger the escape condition to win.

4.2: Controls

Input	Action
W / A / S / D	Move (Up/Left/Down/Right)
Left Shift (hold)	Sprint
I	Open / Close Inventory
Escape / P	Pause Game
Mouse Move	Aim / Rotate Player
R (debug)	Regenerate Map (debug only)

4.3: Survival Stats

The player has four survival stats, all displayed on the HUD as coloured bars in the top-left of the screen:

Stat	Behaviour
Health (Red)	Damage from enemies/hazards; drains if hunger or water is empty.
Stamina (White)	Drains at 15 units/sec while sprinting; regenerates at 5 units/sec otherwise.
Hunger (Orange)	Drains passively over time; triggers health drain when empty.
Water (Blue)	Drains passively over time; triggers health drain when empty.

4.4: Victory and Defeat

Victory: The player collects all required escape components from POIs and activates the escape sequence at the Hideout. (Planned, subject to implementation time.)

Defeat: The player's health reaches zero. The world resets on death, generating a new map with a new seed.

5. The Game World

5.1: Overview

The world of Thorns is a dense, procedurally generated forest that is never the same twice. It is structured as a 128x128 grid of 64x64-pixel tiles, producing a total world space of 8192x8192 pixels. The world is divided into regions using a Voronoi diagram, with Perlin noise layered on top for organic object density variation.

The world feels alive because no two runs share the same layout. Trees, roots, and forest debris are scattered organically using Perlin noise thresholding, while Voronoi regionalization ensures that points of interest are distributed across the map with adequate spacing between them.

5.2 The Physical World

5.2.1: Overview

The terrain is tile-based. Each tile carries a terrain type (Grass, Forest, POI), a walkability flag, and a Voronoi region ID. The tile grid underpins all generation logic, Voronoi sites are positioned in tile space, POIs mark tiles as non-walkable, and object placement checks tile type before spawning.

5.2.2: Key Locations

- Player Hideout: Always placed at the exact centre of the map (tile 64,64). This is the player's starting point and the eventual escape location.
- Farms: Placed at Voronoi sites distributed across the map. Each farm is a procedurally positioned point of interest with collision geometry loaded from a TMX template file.
- Forest Clearings: Open grass areas generated between tree clusters by the Perlin noise threshold system.

5.2.3: Travel

The player navigates on foot using WASD movement. Sprint allows faster traversal at the cost of stamina. There are no fast-travel or teleport mechanics, the world must be explored on foot, creating tension around resource management as distances increase.

5.2.4: Scale

The world is 8192x8192 pixels at the default 128x128 tile map size with 64-pixel tiles. The camera follows the player and is clamped to map bounds so the player cannot see outside the world edge.

5.2.5: Objects

World objects are procedurally placed forest assets drawn from a shared texture atlas. Current types include small roots, large roots, and two tree canopy variants. Each type has polygon collision data loaded from a TMX file and shared across all instances of that type using the Flyweight pattern.

5.2.6: Weather and Day/Night

Weather and day/night cycles are not implemented in the current version. They are identified as future extensions in the Technical Design Document.

5.2.7: Time

The world operates on a day timer. A countdown, visible in the top-right corner of the HUD, begins at the start of each run. When it reaches zero, the map regenerates from a new seed: all enemies despawn, all loot is respawned at fresh positions, and the player is returned to the Hideout at the centre of the new map. The timer resets each regeneration.
This mechanic creates a recurring pressure cycle. The player must assess which POIs to prioritise within each day window, knowing that unexplored areas will be reshuffled when the timer expires. It also prevents indefinite resource farming from a single map layout.
A full day/night visual cycle is not implemented. The timer represents the passage of in-world time without an accompanying lighting change. This is identified as a future extension.

6. Rendering System

6.1: Overview

Thorns uses SFML 3.0 for all 2D rendering. The rendering pipeline separates world-space content from screen-space UI using two distinct SFML views: m_gameView for the game world and m_uiView for the HUD and inventory overlay.

6.2: 2D Rendering

All rendering is 2D and top-down. Tile terrain is rendered using a shared texture atlas with a vertex array for performance. World objects (trees, roots) are rendered using individual sprite components sharing a single atlas texture. POI sprites are layered on top of terrain. The HUD and inventory panel are always rendered in screen space, unaffected by camera movement.

6.3: Camera

6.3.1: Overview

The camera is a simple follow camera implemented via SFML's View system. The game view tracks the player's world-space position and is clamped to the map bounds, so the view never reveals empty space outside the world edge.

6.3.2: Camera Clamping

When the player approaches a map edge, the camera stops following and holds at the boundary. This is calculated by comparing the half-size of the view against the world dimensions and clamping the view centre accordingly.

6.3.3: UI View

A separate fixed view (m_uiView) is always set to the screen resolution (1920x1080 by default). All HUD elements, stat bars, inventory panel, cursor, are rendered under this view so they remain in fixed screen positions regardless of world camera movement.

7. Game Engine

7.1: Overview

Thorns is built on a custom lightweight engine using SFML 3.0. The engine is structured around a fixed-timestep game loop running at 60 updates per second with uncapped rendering. The core systems are the GameStateManager (state stack), InputController (keyboard/gamepad abstraction), CollisionManager (SAT polygon collision), and MapGenerator (procedural world pipeline).

7.2: State Management

A pushdown automaton state stack manages game states: MainMenu, Playing, Paused, Settings, and GameOver. Push transitions preserve the current state (e.g. pausing keeps the game world alive beneath the pause menu). Change transitions replace the stack entirely (e.g. returning to the main menu from gameplay).

7.3: Collision Detection

Collision uses a two-phase approach. AABB (axis-aligned bounding box) checks perform a fast broad-phase cull. If the AABB overlaps, SAT (Separating Axis Theorem) is applied to the object's polygon collision shape for precise narrow-phase resolution. Polygon shapes are stored once per object type in the Flyweight template manager and referenced by non-owning pointers in each instance, avoiding redundant data copies.

7.4: Procedural Generation Pipeline

The map generator runs a multi-phase pipeline on world creation:

- Phase 1 Voronoi Diagram: Poisson disk sampling places Voronoi sites across the map. Each tile is assigned to its nearest site, creating distinct regions. POIs are spawned at selected sites.
- Phase 2 Perlin Noise Object Placement: Octave Perlin noise is sampled at each tile position. Tiles with noise values above a threshold receive a world object (tree, root, etc.). Object type is determined by the noise value range.
- Phases 3 & 4 (Planned, not implemented): Cellular Automata refinement and Dijkstra connectivity check.

8. The World Layout

8.1: Overview

The world layout is fully procedural. No tiles are hand placed. The Voronoi diagram divides the map into organic regions, and POIs are distributed within those regions. The Hideout is always at the map centre, all other POIs are placed at Voronoi sites that meet minimum spacing requirements.

8.1.1: Tile Terrain Types

Terrain Type	Description
Grass	Open walkable area, eligible for object placement.
Forest	Dense forest area (planned for biome system).
POI_Collision	Occupied by a POI footprint, non-walkable.
UNKNOWN	Ungenerated or invalid tile (black in debug mode).

8.1.2: POI Exclusion Zones

Each POI has an exclusion radius that prevents Voronoi sites from being placed too close to it. This ensures POIs are not clustered together and that adequate open space surrounds each location.

9. Game Characters

9.1: Overview

Thorns features one player character and one enemy type currently in development. The player character is fully implemented with movement, rotation, survival stats, inventory, and collision. The enemy is in active development.

9.2: The Player

The player is a top-down character rendered from a sprite atlas. Movement is controlled via WASD with mouse-aimed rotation. The player has four states: Idle, Walk, Sprint, and

InventoryOpen. Sprinting drains stamina, the inventory state locks movement while open. The player entity uses a component-based design, composing SpriteComponent, CursorComponent, Inventory, HealthComponent, StatComponent (x3 for stamina/hunger/water), and HUDComponent.

9.3: Enemies

Two enemy types are present in the world. Both are managed by EnemyManager using an Object Pool, with fixed-size pre-allocated slot arrays for each type. Enemies are spawned at Voronoi site positions on map generation and despawned on map regeneration.

Savage Enemy
The SavageEnemy is a direct pursuer. It has three AI states:
Line of sight is checked each frame using a ray march along the tile grid. A tile that is not walkable (a wall, POI collision zone, or dense object) blocks sight. The Savage Enemy cannot see through forest objects or building walls.

Chomper Enemy
The ChomperEnemy shares the three-state base AI with SavageEnemy but adds a fourth state: Leap. When chasing and within 220px of the player, and the leap cooldown has elapsed, it locks a direction and launches itself at 420px/s for 0.35 seconds. Hitting a wall during the leap cancels it and imposes a 3-second cooldown before the next leap can trigger.

The Leap makes ChomperEnemy harder to evade than SavageEnemy at close range. Players need to maintain distance or break line of sight before the Chomper enters leap range.

10. User Interface

10.1: Overview

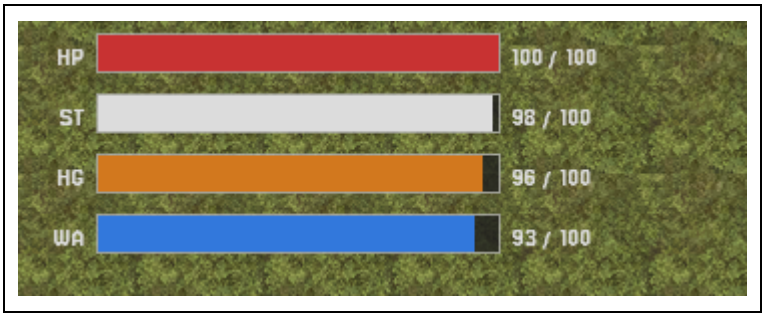
The UI is split into two layers: the in-world HUD (rendered in screen space over the game world) and the menu system (rendered during non-playing states). All UI elements use SFML shapes and text rendered under the dedicated m_uiView to ensure fixed screen-space positioning.

10.2: HUD

The HUD is displayed during gameplay and shows four stat bars in the top-left corner of the screen:

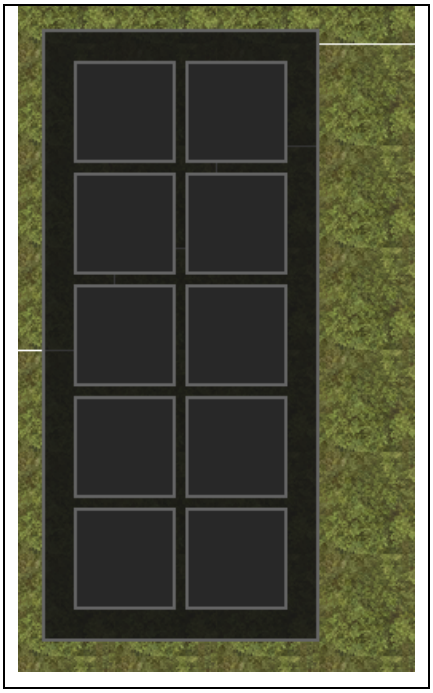
- Health bar (Red): current health as a ratio of maximum health.
- Stamina bar (White): current stamina.
- Hunger bar (Orange): current hunger level.
- Water bar (Blue): current water level.

Each bar is labelled with the stat name and displays the current numeric value. Bars are rendered using a generic drawBar helper inside HUDComponent.



A day timer panel is displayed in the top-right corner of the screen during gameplay and while paused. It shows a label (DAY ENDS IN). When less than ten seconds remain, the countdown text turns red as a visual warning.

10.3: Inventory



The inventory panel (10 slots arranged in a 2x5 grid) is toggled with the “I” key and anchored to the left side of the screen. When open, the player cannot move. Item slots display item name and quantity when filled. The panel uses ScreenSize::s_width to anchor correctly regardless of resolution changes.

10.4: Menus

Three menus are implemented: Main Menu (start, settings, quit), Pause Menu (resume, settings, back to main menu), and Settings Menu (key rebinding, resolution selection, fullscreen toggle). Menus use the ButtonComponent class and support both mouse and keyboard/gamepad navigation.

11. Weapons

11.1: Overview

Three weapon types are available as world items, collectible from the map and usable from the inventory. All weapons are triggered by left mouse button during gameplay. Hit detection is handled by EnemyManager::checkAttackHit, which is called from Game with the player position, mouse world position, weapon type, and damage value.

Weapon	Type	Range	Damage	Hit Method
Knife	Melee	80px radius	To be balanced	Radius check around player position
Axe	Melee	80px radius	To be balanced	Radius check around player position
Gun	Ranged	800px	To be balanced	Ray projection from player to cursor, enemies within 40px of the ray are hit

Melee weapons (Knife, Axe) hit any active enemy within the radius regardless of direction. Ranged (Gun) casts a ray from the player position toward the mouse world position and tests each enemy's perpendicular distance from that ray against a 40px hit threshold, simulating a narrow projectile path.

Weapons are collected from the world as WorldItem instances and added to the inventory as equippable items. The ItemUseCategory enum distinguishes consumables (Food, Water, FirstAid, Bandage) from equippables (Knife, Axe, Gun) in the item type registry.

Attack animation and visual feedback are not yet implemented. This is identified as future work.

12. Musical Scores and Sound Effects

12.1: Overview

Audio is not implemented in the current version. The SFML audio module is included as a project dependency. The Assets namespace in AssetPaths.h contains a reserved Audio section for future sound file paths. Planned audio features include ambient forest sound, footstep variation by terrain type, and UI interaction sounds.

13. Single-Player Game

13.1: Overview

Thorns is a single-player-only game. There is no network code or multiplayer mode.

13.1.1: Story

The player wakes in an unfamiliar forest clearing. The trees are wrong, too dense, too uniform, repeating in patterns that should not occur in nature. There are no roads, no signs, no people. Only the ruins of a Hideout at the centre of the forest, and fragments of machinery scattered at outlying farms. The forest loops. The only way out is to find the pieces and build the escape.

13.1.2: Hours of Gameplay

An average run is estimated at 10 to 20 minutes depending on world seed, player familiarity, and survival management skill. The procedural world generation ensures replayability, as each run presents a unique map layout.

13.1.3: Victory Conditions

The player wins by collecting all required escape components from POIs across the map and activating the escape mechanism at the Hideout. If the player's health reaches zero before escaping, the run ends and a new world is generated.

14. Character Rendering

14.1: Overview

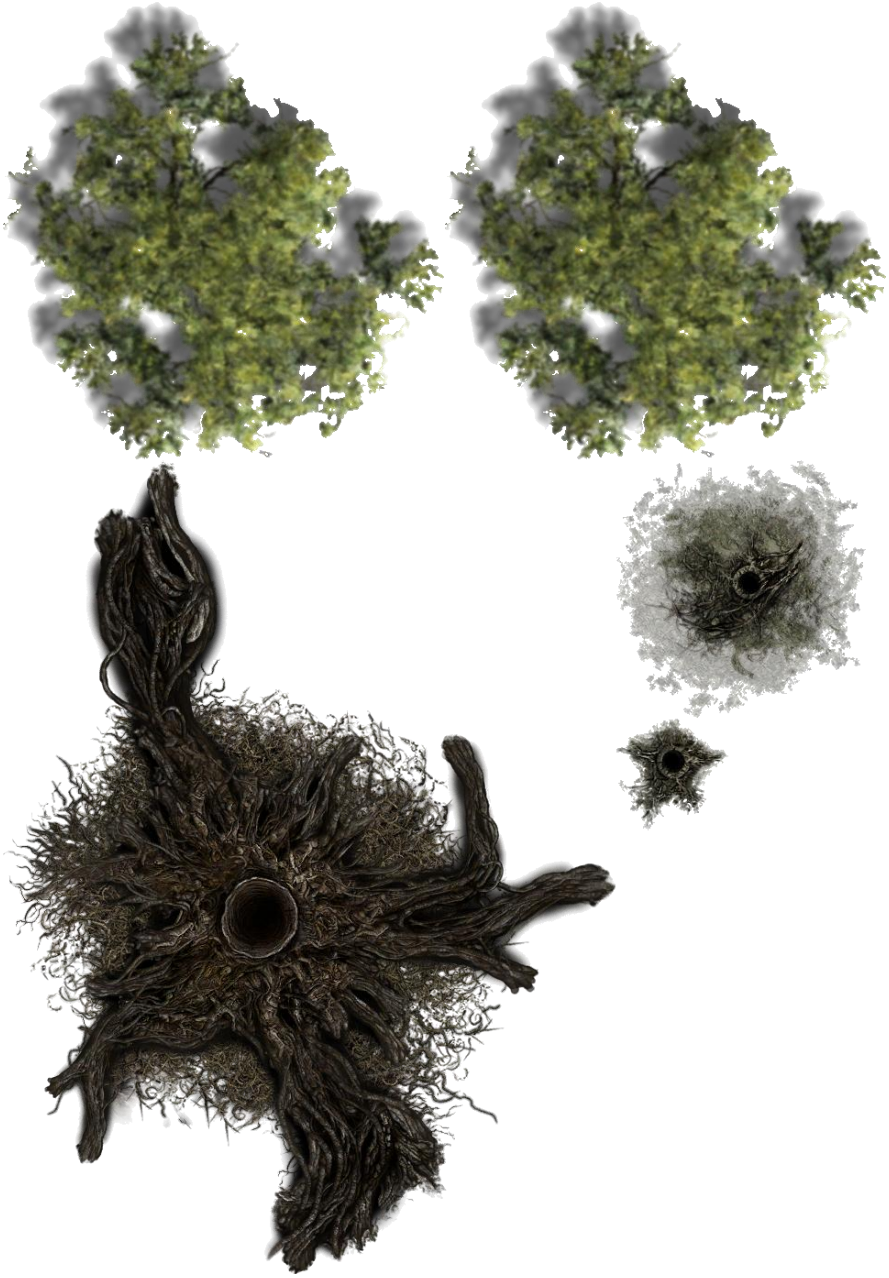
All characters are rendered as 2D sprites drawn from texture atlases using SFML. The SpriteComponent class wraps the SFML Sprite, managing texture loading, atlas region selection, scaling, rotation, and origin management.

14.1.1: Player Rendering

The player sprite is loaded from a player atlas (player_atlas.png). The sprite origin is centred so that rotation correctly pivots around the character's centre. The player always faces the mouse cursor position, with rotation calculated from the world-space vector between the player position and the mouse world position.

14.1.2: World Object Rendering

World objects (trees, roots) share a single atlas texture loaded once into memory. Each WorldObject instance holds a SpriteComponent configured with the appropriate atlas region. Rendering is frustum-culled, only objects within the current camera view are drawn each frame.



15. References

[1] Nystrom, R. (2014). Game Programming Patterns. [Online]. Available at: https://gameprogrammingpatterns.com/

[2] Perlin, K. (2002). Improving Noise. In Proceedings of SIGGRAPH 2002. ACM Press. Available at: http://mrl.nyu.edu/~perlin/paper445.pdf

[3] Bridson, R. (2007). Fast Poisson Disk Sampling in Arbitrary Dimensions. ACM SIGGRAPH 2007 Sketches. Available at: https://dl.acm.org/doi/10.1145/1278780.1278807

[4] Solarian Programmer. (2012). Perlin Noise in C++11. [Online]. Available at: https://solarianprogrammer.com/2012/07/18/perlin-noise-cpp-11/

[5] leatherbee.org. (2018). Terrain Generation 3 Voronoi Diagrams. [Online]. Available at: https://leatherbee.org/index.php/2018/10/06/terrain-generation-3-voronoi-diagrams/

[6] GeeksforGeeks. Voronoi Diagram. [Online]. Available at: https://www.geeksforgeeks.org/dsa/voronoi-diagram/

[7] Cherno. (2022). std::unordered_map in C++. [Online]. Available at: https://youtu.be/KiB0vRi2wlc?si=c__NeniIIIjXwTsD

[8] Pezzotti, J. (2020). Poisson Disk Sampling. [Online Video]. Available at: https://youtu.be/jofUe3Kjag4?si=HnIRv3t2Yv7CT_Gq

[9] SFML Development Team. (2024). SFML 3.0 Documentation. [Online]. Available at: https://www.sfml-dev.org/

[10] tmxlite Contributors. tmxlite Library. [Online]. Available at: https://github.com/fallahn/tmxlite